

AGENTIC DECEPTIVE-UI REMEDIATION

Catch. Fix.
Prove.

PRAMAAN catches deceptive interface patterns in frontend code, lets a bounded agent investigate and propose repairs, then makes the deterministic engine prove whether the fix actually worked.

DETECT

INVESTIGATE

VERIFY

One run → source evidence → bounded patch → G1–G5 verification → tamper-evident pack

PRAMAAN

AuditMethodEvidenceDocsENGINE READYStart an audit

Agentic dark-pattern audit

Meet PRAMAAN.
It reads your interface *the way a regulator would* — then fixes what it finds, and proves it.

Give it a frontend. PRAMAAN finds deceptive UI patterns, proposes bounded fixes, verifies every change with a deterministic engine, and leaves the evidence behind.

Start an auditWatch the demoSee how verification worksVerify an evidence pack

`npx pramaan audit ./src`

The AI proposes. The engine decides.

01 — PROBLEM

Deceptive UI is easy to ship. Hard to prove you fixed.

Most tools stop at “flagged.” PRAMAAN closes the loop from source code to verified evidence.

Hidden in implementation

Preselected add-ons, false urgency, interface interference and late-fee disclosure can live in ordinary frontend code — not in a neat compliance checklist.

AI alone is not trustworthy

A model can suggest a plausible repair, but it can also hallucinate, over-edit, or claim success without deterministic proof.

Evidence gets lost

Teams need the exact finding, source context, patch, verification gates and resulting proof chain — not just a dashboard score.

THE GAP

Detection → remediation → verification → evidence

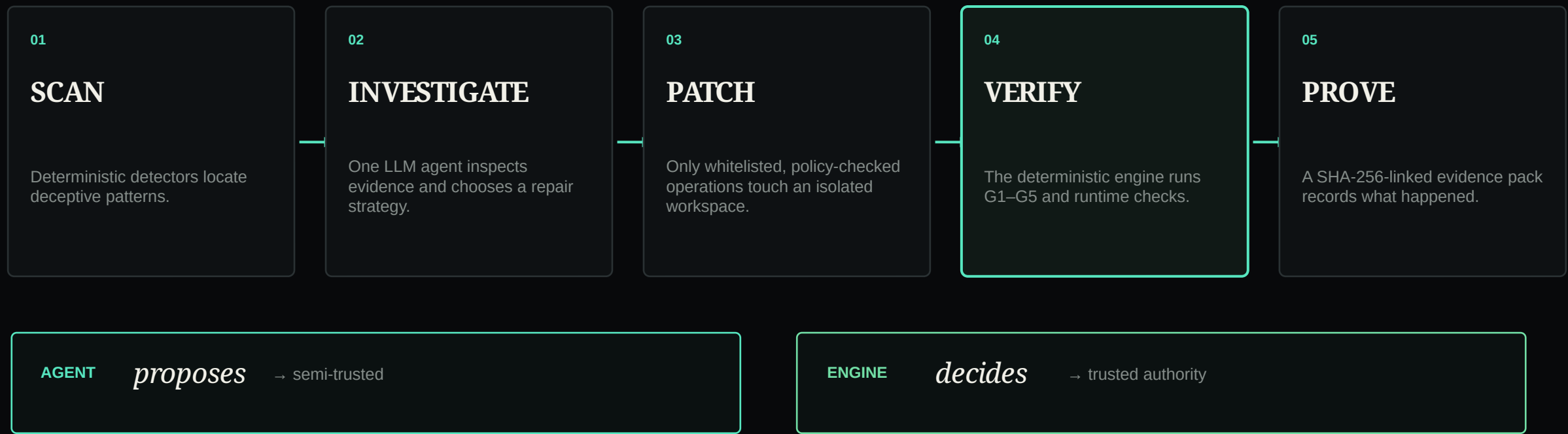
PRAMAAN is built around that complete chain.

Not a chatbot. Not a crawler. Not a legal certifier.

02 — HOW IT WORKS

One loop. Two authorities.

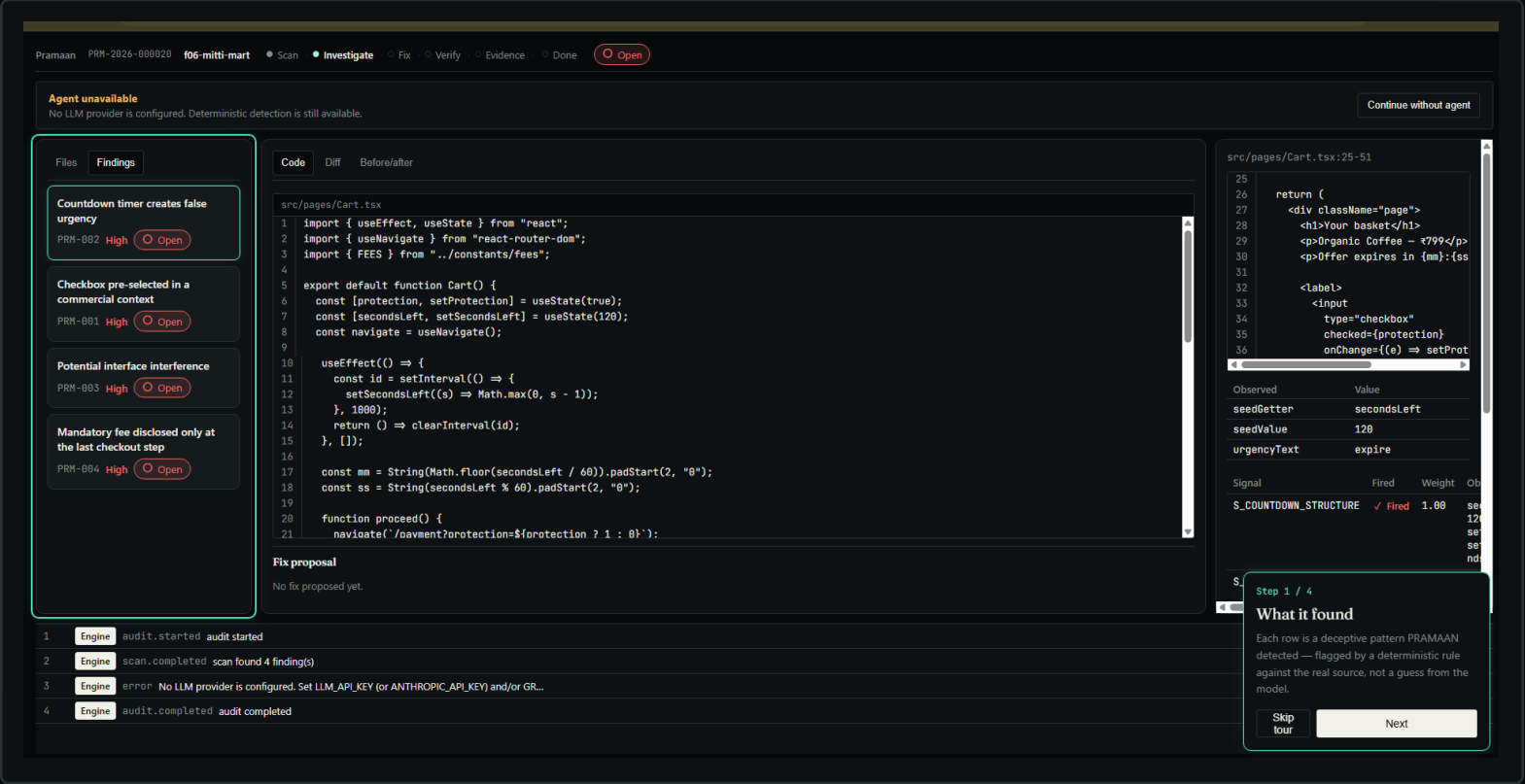
The agent is useful because it is bounded. The system is trustworthy because verification stays deterministic.



03 — WORKING PRODUCT

This is already running end-to-end — except the live model smoke test.

The engine, server, fixtures and UI are real; the only remaining risk is the live LLM tool-calling path.



PROOF, NOT CLAIMS

- 5 detector families
- 7 whitelisted patch ops
- 106 tests passing
- 13/13 fixtures build clean
- G1-G5 verification gates
- SHA-256 evidence chain

Remaining kill test Live LLM → tool call → proposal → patch → deterministic re-verification

WHY PRAMAAN

From manipulative UI to verifiable proof.

Most products either detect, generate, or report. PRAMAAN closes the loop — and keeps authority where it belongs.

Explainable

Every finding keeps source evidence, rule provenance and a visible verification trail.

Agentic, but bounded

The model investigates and proposes. Policy and deterministic gates control what actually lands.

Demoable today

Prepared fixtures, real path-based audits, live trace, evidence pack verification and a judge-friendly walkthrough.

SCAN → INVESTIGATE → FIX → VERIFY → PROVE